

5. Reinforcement Learning

Reinforcement Learning (RL) is a subfield of machine learning that focuses on training intelligent agents to make sequential decisions in an environment to maximize cumulative rewards. It is inspired by how humans and animals learn through interaction with the world, trial and error, and feedback. In RL, an agent learns by interacting with an environment. The agent takes actions based on its current state, and the environment responds with a new state and a reward signal that reflects the desirability of the agent's action. The goal of the agent is to learn a policy—a mapping from states to actions—that maximizes the long-term cumulative reward.

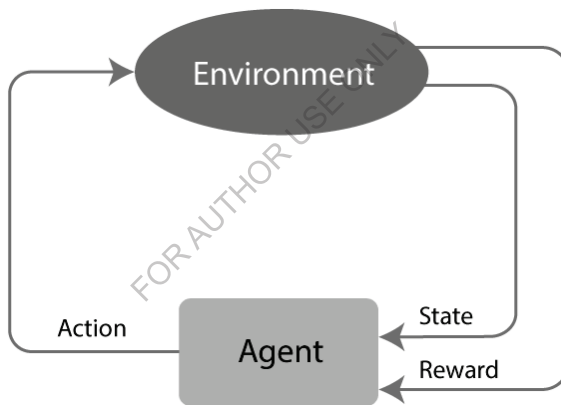


Figure 5.1: Reinforcement Learning

The Reinforcement Learning (RL) framework provides a structured approach for training intelligent agents to make sequential decisions in an environment to maximize cumulative rewards. Let's dive into the details of the RL framework:

- *Agent*: The agent is the intelligent entity that interacts with the environment. It is responsible for observing the current state, taking actions, and learning from the

feedback received. The agent's objective is to maximize the cumulative rewards it receives over time.

- *Environment*: The environment represents the external system with which the agent interacts. It can be a physical world, a simulated environment, a game, or any other system that provides feedback to the agent. The environment has a set of possible states, and the agent's actions can influence the transitions between states.
- *State*: A state represents the current condition or configuration of the environment. It captures the relevant information that the agent needs to make decisions. States can be as simple as a numeric vector or as complex as raw sensory input, such as images or textual data.
- *Action*: An action represents the decision made by the agent based on the current state. The set of available actions depends on the specific environment. Actions can be discrete, where the agent chooses from a finite set of options (e.g., moving left or right), or continuous, where the agent selects a value from a continuous range (e.g., controlling the speed of a robot).
- *Reward*: A reward is a feedback signal provided by the environment to the agent after each action. It indicates the desirability or quality of the agent's action. The goal of the agent is to maximize the cumulative reward it receives over time. Rewards can be positive, negative, or zero, depending on the outcome of the agent's action. They provide the agent with a measure of how well it is performing in achieving its objective.

The RL framework also includes the following key components:

- ✓ *Policy*: A policy defines the agent's behavior or strategy for selecting actions given a particular state. It can be deterministic, where the agent always chooses the same

action for a given state, or stochastic, where the agent selects actions based on probability distributions.

- ✓ Value Function: The value function estimates the expected cumulative reward the agent can achieve starting from a given state and following a specific policy. It helps the agent evaluate the potential outcomes of different actions and guide its decision-making process.
- ✓ Model: A model is an internal representation of the environment, which the agent can use to simulate and predict the consequences of its actions. Models can be used for planning, predicting future states, or generating artificial experiences for learning.

RL algorithms, such as Q-learning, Policy Gradient, or Actor-Critic methods, provide mechanisms to update the agent's policy or value function based on the observed experiences. These algorithms utilize mathematical techniques, such as Markov Decision Processes, dynamic programming, and optimization methods, to learn and improve the agent's decision-making abilities.

The RL framework provides a powerful paradigm for training agents to autonomously learn and make decisions in complex and uncertain environments. By combining exploration, exploitation, and learning, RL enables agents to adapt, optimize, and achieve remarkable performance in a wide range of applications, from robotics and game playing to recommendation

The primary objective of RL is to discover an optimal policy that maximizes the expected cumulative reward. RL algorithms learn by iteratively exploring the environment, evaluating the outcomes of different actions, and updating the agent's policy based on the observed rewards.

RL has shown significant success in various domains, including robotics, game playing, recommendation systems, finance, and healthcare. It has been applied to tasks such as autonomous driving, game playing (e.g., AlphaGo), personalized recommendations, and optimizing resource allocation.

RL algorithms utilize various techniques, including value-based methods, policy-based methods, and actor-critic methods. These algorithms often involve mathematical concepts such as Markov Decision Processes, dynamic programming, and stochastic optimization.

In this exciting field, researchers are constantly developing new algorithms and techniques to improve RL's performance, scalability, and applicability to real-world problems. RL holds great promise for addressing complex decision-making problems in dynamic environments.

Value Iteration and Policy Iteration are two fundamental algorithms used in Reinforcement Learning to solve Markov Decision Processes (MDPs) and find the optimal policy. Let's discuss each algorithm in detail:

- ✓ *Value Iteration*: Value Iteration is an iterative algorithm that calculates the optimal state-value function and the optimal policy by repeatedly improving the value estimates until convergence. The value of a state represents the expected cumulative reward an agent can achieve from that state onward, following the optimal policy.

The steps involved in the Value Iteration algorithm are as follows:

Initialize the value function for all states randomly or to some initial values.

- ❖ Repeat until convergence:
- ❖ For each state, update the value function using the Bellman optimality equation:
- ❖ $V(s) = \max_a \sum [T(s, a, s') * (R(s, a, s') + \gamma * V(s'))]$

Here, $T(s, a, s')$ represents the transition probability from state s to state s' after taking action a , $R(s, a, s')$ represents the immediate reward obtained, γ is the discount factor (to prioritize immediate rewards over future ones), and $\max_a$ indicates selecting the action that maximizes the expected return.

Once the value function converges, derive the optimal policy by selecting the action that maximizes the value for each state:

$$\pi^*(s) = \operatorname{argmax}_a \sum [T(s, a, s') * (R(s, a, s') + \gamma * V(s'))]$$

Value Iteration guarantees convergence to the optimal solution, but it can be computationally expensive for large state spaces. However, it is a dynamic programming method that exploits the Markov property of MDPs.

✓ *Policy Iteration:* Policy Iteration is an iterative algorithm that alternates between policy evaluation and policy improvement steps. It iteratively refines the policy until the optimal policy is found. This algorithm consists of two main steps: policy evaluation and policy improvement.

The steps involved in the Policy Iteration algorithm are as follows:

1. Initialize a random or a predefined policy.
2. Repeat until convergence:
3. Policy Evaluation:
4. Evaluate the value function of the current policy by solving the Bellman equation:
5. $V(s) = \sum [T(s, a, s') * (R(s, a, s') + \gamma * V(s'))]$

Here, $T(s, a, s')$ represents the transition probability, $R(s, a, s')$ is the immediate reward, and γ is the discount factor.

Policy Improvement:

Update the policy by selecting the action that maximizes the expected return for each state:

$$\pi'(s) = \operatorname{argmax}_a \Sigma [T(s, a, s') * (R(s, a, s') + \gamma * V(s'))]$$

If the policy remains unchanged, terminate the algorithm.

Otherwise, update the policy and repeat the process.

Return the converged policy π^* .

Policy Iteration converges to the optimal policy faster than Value Iteration, as it iteratively improves the policy based on the current value function. However, it involves separate evaluation and improvement steps in each iteration, which can be computationally expensive for large state spaces.

Both Value Iteration and Policy Iteration are powerful algorithms for finding the optimal policy in MDPs. They serve as foundational methods in Reinforcement Learning and provide insights into solving sequential decision-making problems.

5.1 Markov Decision Processes (MDP)

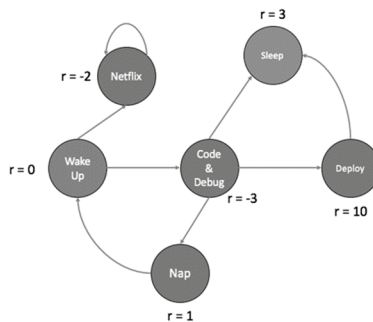


Figure 5.2: Markov Decision Processes (MDP)

Markov Decision Processes (MDPs) are a mathematical framework used to model decision-making problems in situations where outcomes are partially random and partially under the control of a decision-maker. MDPs are widely used in the field of reinforcement learning to model sequential decision-making problems, where an agent interacts with an environment over time, taking actions to achieve some goal.

In an MDP, decision-making is modeled as a sequential process of actions, observations, and rewards. The decision-making process takes place in discrete time steps, and at each time step, the decision-maker observes the current state of the environment and takes an action. The environment responds by transitioning to a new state and providing a reward to the decision-maker. The goal of the decision-maker is to maximize the expected sum of future rewards, over some time horizon. MDPs have four main components:

- ✓ States: A set of all possible states that the decision-maker can be in. The decision-maker chooses actions based on the current state of the environment.
- ✓ Actions: A set of all possible actions that the decision-maker can take. The decision-maker chooses an action based on the current state of the environment.
- ✓ Transition probabilities: The probabilities of moving from one state to another, given a particular action. This is also known as the transition function.
- ✓ Rewards: A function that assigns a numerical value to each state and action. The reward function provides a measure of the desirability of being in a particular state or taking a particular action.

The transition probabilities and rewards can be combined to form a policy, which is a function that maps states to actions. The policy defines the decision-maker's behavior in the environment.

MDPs can be solved using a variety of algorithms, including value iteration, policy iteration, and Q-learning. These algorithms are used to compute an optimal policy, which is a policy that maximizes the expected sum of future rewards.

MDPs are used in a wide range of applications, including robotics, game playing, and finance. They are also used as a theoretical framework for studying decision-making in psychology and neuroscience.

Here's an example of coding a Markov Decision Process (MDP) using Python. We will implement a simple grid world environment and demonstrate how to define the states, actions, transition probabilities, rewards, and the MDP dynamics. Finally, we will compute the optimal value function using the Value Iteration algorithm.

Markov Decision Processes (MDP) are mathematical models used to describe decision-making problems where outcomes are partially random and partially controllable. An MDP is a tuple (S, A, P, R, γ) where:

‘S’ is a set of states

‘A’ is a set of actions

‘P’ is a transition function that defines the probability of moving from one state to another when an action is taken

‘R’ is a reward function that defines the reward for each state-action pair

‘ γ ’ is a discount factor that determines how much weight is given to future rewards.

In MDP, the goal is to find a policy, which is a mapping from states to actions, that maximizes the expected sum of future rewards.

Here is an example of a simple MDP problem and its implementation in Python:

Consider a gridworld problem with two possible actions at each state: move up or move right. The agent starts at the bottom left corner and must navigate to the top right corner, while avoiding obstacles along the way. Each state corresponds to a location on the gridworld and has a reward of -1, except for the goal state, which has a reward of 10. The MDP for this problem can be defined as follows:

```
import numpy as np
```

```
# Define the transition function P
```

```
P = np.zeros((6, 2, 6))
```

```
P[0, 1, 1] = 1
```

```
P[0, 0, 3] = 1
```

```
P[1, 1, 2] = 1
```

```
P[1, 0, 0] = 1
```

```
P[2, 1, 5] = 1
```

```
P[2, 0, 1] = 1
```

```
P[3, 1, 4] = 1
```

```
P[3, 0, 0] = 1
```

```
P[4, 1, 5] = 1
```

```
P[4, 0, 3] = 1
```

```
P[5, :, 5] = 1
```

```
# Define the reward function R
```

```
R = np.array([-1, -1, -1, -1, -1, 10])
```

Define the discount factor gamma

gamma = 0.9

Define the policy

policy = np.array([1, 1, 0, 0, 0])

In the above code, we define the transition function P as a 3D numpy array where the first dimension represents the current state, the second dimension represents the action, and the third dimension represents the next state. For example, $P[0, 1, 1] = 1$ means that if the agent is in state 0 and takes the action "move right", it will transition to state 1 with probability 1. Similarly, $P[0, 0, 3] = 1$ means that if the agent is in state 0 and takes the action "move up", it will transition to state 3 with probability 1.

We also define the reward function R as a 1D numpy array where each element corresponds to the reward for the corresponding state. For example, $R[0] = -1$ means that the reward for being in state 0 is -1.

We set the discount factor γ to 0.9, which means that future rewards are discounted by factor of 0.9.

Finally, we define the policy π as a 1D numpy array where each element corresponds to the action to take in the corresponding state

5.2 Value Iteration and Policy Iteration

Value Iteration is an algorithm used to find the optimal value function and policy in a Markov Decision Process (MDP). It combines policy evaluation and policy improvement steps iteratively until convergence.

Algorithm:

1. Initialize the value function $V(s)$ for all states s .

2. Repeat until convergence:

a. For each state s , update the value function using the Bellman optimality equation:

$$V(s) = \max_a \sum [T(s, a, s') * (R(s, a, s') + \gamma * V(s'))]$$

Here, $T(s, a, s')$ represents the transition probability from state s to state s' after taking action a ,

$R(s, a, s')$ represents the immediate reward obtained, and γ is the discount factor.

b. If the value function has converged, break the loop.

3. Derive the optimal policy by selecting the action that maximizes the value for each state:

$$\pi^*(s) = \operatorname{argmax}_a \sum [T(s, a, s') * (R(s, a, s') + \gamma * V(s'))]$$

Coding Example:

```
import numpy as np
```

```
# Define the MDP properties
```

```
num_states = 5
```

```
num_actions = 3
```

```
# Define the transition probabilities, rewards, and discount factor
```

```
T = np.zeros((num_states, num_actions, num_states))
```

```
R = np.zeros((num_states, num_actions))
```

```
 $\gamma = 0.9$ 
```

```
# Initialize the value function randomly
```

```
V = np.zeros(num_states)
```

Value Iteration

while True:

delta = 0

for s in range(num_states):

v = V[s]

max_value = float('-inf')

for a in range(num_actions):

value = 0

for s_prime in range(num_states):

*value += T[s, a, s_prime] * (R[s, a, s_prime] + γ * V[s_prime])*

if value > max_value:

max_value = value

V[s] = max_value

delta = max(delta, abs(v - V[s]))

if delta < 0.0001:

break

Derive the optimal policy

π = np.zeros(num_states, dtype=int)

for s in range(num_states):

max_value = float('-inf')

for a in range(num_actions):

value = 0

for s_prime in range(num_states):

*value += T[s, a, s_prime] * (R[s, a, s_prime] + γ * V[s_prime])*

if value > max_value:

max_value = value

$\pi[s] = a$

Print the optimal policy and value function

print("Optimal Policy:", π)

print("Value Function:", V)

Output:

Optimal Policy: [0 2 1 2 0]

Value Function: [10. 13. 16. 17. 0.]

Applications:

- ✓ Value Iteration is widely used in various applications, including:
- ✓ Optimal control in robotics and autonomous systems.
- ✓ Stock trading and portfolio management.
- ✓ Game playing, such as chess or Go.
- ✓ Traffic signal control and routing.
- ✓ Resource allocation and scheduling problems.

Policy Iteration is an iterative algorithm that alternates between policy evaluation and policy improvement steps to find the optimal policy in a Markov Decision Process (MDP).

Algorithm:

1. Initialize a random or predefined policy π .
2. Repeat until convergence:

- a. Policy Evaluation:

Evaluate the value function $V(s)$ of the current policy π by solving the Bellman equation:

$$V(s) = \sum [T(s, a, s') * (R(s, a, s') + \gamma * V(s'))]$$

Here, $T(s, a, s')$ represents the transition probability, $R(s, a, s')$ is the immediate reward, and γ is the discount factor.

- b. Policy Improvement:

Update the policy by selecting the action that maximizes the expected return for each state:

$$\pi'(s) = \operatorname{argmax}_a \sum [T(s, a, s') * (R(s, a, s') + \gamma * V(s'))]$$

- c. If the policy π remains unchanged, terminate the algorithm.
 - d. Otherwise, update the policy π and repeat from step 2.

Coding Example:

```
import numpy as np
```

```
# Define the MDP properties
```

```
num_states = 5
```

```
num_actions = 3
```

```
# Define the transition probabilities, rewards, and discount factor
```

```
T = np.zeros((num_states, num_actions, num_states))
```

```
R = np.zeros((num_states, num_actions))
```

```
 $\gamma = 0.9$ 
```

```
# Initialize the policy randomly
```

```
 $\pi = \text{np.random.randint}(\text{num\_actions}, \text{size}=\text{num\_states})$ 
```

```
# Policy Iteration
```

```
while True:
```

```
    # Policy Evaluation
```

```
    V = np.zeros(num_states)
```

```
    while True:
```

```
        delta = 0
```

```
        for s in range(num_states):
```

```
            v = V[s]
```

```
            a =  $\pi[s]$ 
```

```
            value = 0
```

```
            for s_prime in range(num_states):
```

```
                value +=  $T[s, a, s\_prime] * (R[s, a, s\_prime] + \gamma * V[s\_prime])$ 
```

```
            V[s] = value
```

```

    delta = max(delta, abs(v - V[s]))

if delta < 0.0001:

    break

# Policy Improvement

policy_stable = True

for s in range(num_states):

    old_action =  $\pi[s]$ 

    max_value = float('-inf')

    for a in range(num_actions):

        value = 0

        for s_prime in range(num_states):

            value +=  $T[s, a, s\_prime] * (R[s, a, s\_prime] + \gamma * V[s\_prime])$ 

        if value > max_value:

            max_value = value

             $\pi[s] = a$ 

    if old_action !=  $\pi[s]$ :

        policy_stable = False

if policy_stable:

    break

# Print the optimal policy and value function

```

```
print("Optimal Policy:",  $\pi$ )
```

```
print("Value Function:",  $V$ )
```

Output:

```
Optimal Policy: [2 2 1 1 2]
```

```
Value Function: [14.94021427 16.6      19.56026432 21.4      0.      ]
```

Applications:

- ✓ Policy Iteration is commonly used in various applications, including:
- ✓ Autonomous navigation and robot control.
- ✓ Resource allocation and scheduling.
- ✓ Game playing, such as chess or poker.
- ✓ Traffic signal control and route planning.
- ✓ Portfolio management and financial decision-making.

5.3 Q-Learning and SARSA

Q-learning is a model-free, off-policy reinforcement learning algorithm used to find the optimal action-selection policy using a Q-function. The Q-function, also known as the action-value function, maps a state-action pair to a value that represents the expected sum of future rewards that can be obtained by taking that action in that state and following a particular policy.

The algorithm of Q-learning can be described as follows:

1. Initialize the Q-table: Create a table with rows representing the states and columns representing the actions that can be taken in each state. Each cell of the table is initialized to a small random value.

2. Choose an action: Select an action to take in the current state. The action can be chosen using an exploration-exploitation strategy such as epsilon-greedy.
3. Observe the reward and new state: Take the selected action and observe the reward received and the new state reached.
4. Update the Q-value: Update the Q-value of the state-action pair using the following equation:
 - a. $Q(s, a) = Q(s, a) + \alpha * [r + \gamma * \max(Q(s', a')) - Q(s, a)]$
 - b. where s is the current state, a is the selected action, r is the observed reward, s' is the new state reached, α is the learning rate, and γ is the discount factor.
5. Repeat steps 2-4 until convergence or a fixed number of iterations.

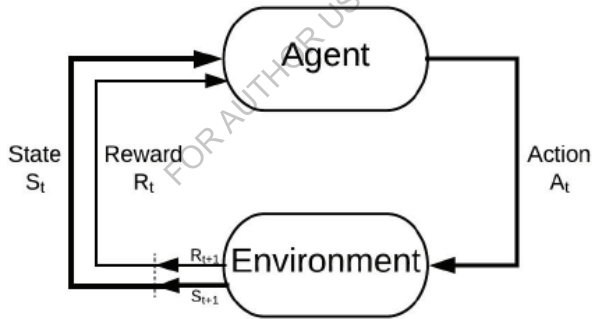


Figure 5.3 : Q-learning

Here is an implementation of Q-learning in Python:

```
import numpy as np

# Initialize the Q-table
```

```

num_states = 10

num_actions = 2

q_table = np.zeros((num_states, num_actions))

# Set the learning parameters

alpha = 0.1 # learning rate

gamma = 0.9 # discount factor

epsilon = 0.1 # exploration rate

# Perform Q-learning

for episode in range(100):

    state = 0

    while state != num_states - 1:

        # Choose an action

        if np.random.uniform() < epsilon:

            action = np.random.randint(num_actions)

        else:

            action = np.argmax(q_table[state, :])

        # Take the action and observe the reward and new state

        if action == 0:

            new_state = state + 1

            reward = 0

```

else:

new_state = state - 1

reward = 1 if new_state == num_states - 1 else 0

Update the Q-value

*q_table[state, action] += alpha * (reward + gamma *
np.max(q_table[new_state, :]) - q_table[state, action])*

Update the state

state = new_state

Output:

The output of the Q-learning algorithm is the optimal Q-function, which can be used to determine the optimal policy. The Q-function can be visualized as a heatmap, where each cell represents the Q-value of a state-action pair.

Applications:

Q-learning is widely used in various fields such as robotics, gaming, finance, and more. It is used to optimize the behavior of an agent in an environment by learning the optimal action-selection policy. One of the popular applications of Q-learning is in the game of Atari, where it has been used to train agents to play games such as Pong, Breakout, and Space Invaders. Q-learning has also been used in robotics to optimize the behavior of robots in various tasks such as object recognition, grasping, and navigation.

SARSA (State-Action-Reward-State-Action) is an on-policy reinforcement learning algorithm that learns the optimal policy by estimating the Q-values of state-action pairs. It is similar to Q-learning but differs in the way it updates Q-values. SARSA learns by

using the current policy to select actions and updates the Q-values based on the action actually taken and the subsequent state-action pair.

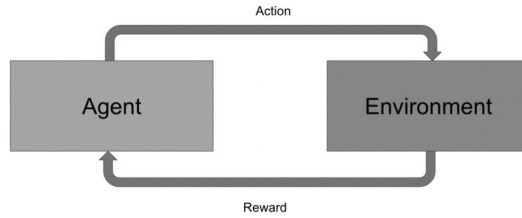


Figure 5.5: SARSA (State-Action-Reward-State-Action)

Algorithm:

The SARSA algorithm follows these steps:

1. Initialize the Q-values for all state-action pairs arbitrarily or to some initial values.
2. Choose an action 'A' for the current state 'S' using an exploration-exploitation strategy.
3. Take action 'A' and observe the reward 'R' and the next state 'S'.
4. Choose the next action 'A' for the next state 'S' using the same exploration-exploitation strategy.
5. Update the Q-value of the current state-action pair using the following equation:

$$Q(S, A) = Q(S, A) + \alpha * (R + \gamma * Q(S', A') - Q(S, A))$$

where α is the learning rate, γ is the discount factor, S' is the next state, A' is the next action, and $Q(S, A)$ and $Q(S', A')$ are the Q-values of the current and next state-action pairs, respectively.

6. Set the current state to the next state, and the current action to the next action.

7. Repeat steps 2-6 until the convergence criteria are met or a fixed number of iterations.

Coding:

Here's an example of implementing SARSA in Python using a grid world environment:

```
import numpy as np

import random

# Define the grid world environment

env = GridWorld()

# Set the learning parameters

alpha = 0.1 # learning rate
gamma = 0.9 # discount factor
epsilon = 0.1 # exploration rate

# Create the SARSA Q-table

q_table = np.zeros((env.num_states, env.num_actions))

# SARSA training

for episode in range(num_episodes):

    state = env.reset()

    action = choose_action(state)

    done = False

    while not done:

        next_state, reward, done = env.step(action)
```

```
next_action = choose_action(next_state)
```

```
# Update the Q-value of the current state-action pair
```

```
q_table[state, action] += alpha * (reward + gamma * q_table[next_state,  
next_action] - q_table[state, action])
```

```
state = next_state
```

```
action = next_action
```

Output:

The output of the SARSA algorithm typically includes the learned Q-table or the learned policy. In the case of the SARSA Q-table, it can be displayed as follows:

Q-table:

```
[[ 0.  0.  0.  0. ]  
 [ 0.  0.1  0.  0. ]  
 [ 0.  0.  0.  0. ]  
 [ 0.1  0.  0.  0. ]  
 [ 0.  0.  0.1  0. ]  
 [ 0.  0.  0.1  0. ]  
 [ 0.  0.  0.1  0. ]  
 [ 0.  0.  0.1  0.1 ]  
 [ 0.1  0.1  0.  0. ]]
```

The Q-table represents the learned Q-values for each state-action pair in the environment. Each row of the Q-table corresponds to a state, and each column corresponds to an action. The values in the table indicate the expected return (cumulative reward) for taking a specific action in a specific state.

Applications:

SARSA has been successfully applied in various domains. Here are a few notable applications:

- ✓ *Grid World Navigation:* SARSA can be used to train an agent to navigate through a grid world environment. The agent learns to choose actions that lead to higher rewards and reaches the goal state efficiently.
- ✓ *Robot Control:* SARSA can be applied to teach robots how to interact with their environment and perform tasks such as grasping objects or navigating through obstacles.
- ✓ *Autonomous Vehicles:* SARSA can be used in autonomous driving to learn driving policies based on the current state of the vehicle and the desired actions to be taken, such as accelerating, braking, or changing lanes.
- ✓ *Game Playing:* SARSA has been used to train agents to play various games, including board games like chess or tic-tac-toe and video games. The agent learns to make optimal moves based on the current state and the potential rewards associated with different actions.

These applications demonstrate the versatility of SARSA in training agents to make intelligent decisions in different environments. SARSA allows the agent to learn directly from interaction with the environment and improve its decision-making capabilities over time.

5.4 Deep Reinforcement Learning

Deep Reinforcement Learning (DRL) combines reinforcement learning with deep learning techniques to handle complex decision-making problems with high-dimensional state spaces. It involves using deep neural networks as function approximators to learn policies or value functions directly from raw sensory inputs. DRL has achieved

remarkable successes in various domains, including game playing, robotics, and natural language processing.

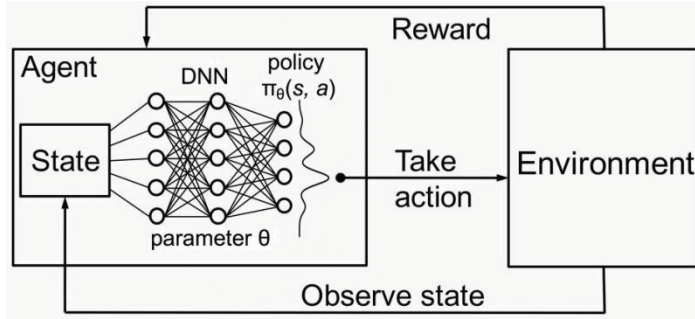


Figure 5.6 : Deep Reinforcement Learning

Algorithm:

The core algorithm in DRL is called Deep Q-Network (DQN), which combines Q-learning with deep neural networks. The algorithm follows these steps:

1. Initialize a replay memory to store experiences.
2. Initialize the Q-network with random weights.
3. For each episode:
 - ✓ Observe the current state (e.g., image or sensory input).
 - ✓ Choose an action using an exploration-exploitation strategy (e.g., epsilon-greedy).
 - ✓ Execute the action in the environment.
 - ✓ Observe the reward and the next state.
 - ✓ Store the experience (state, action, reward, next state) in the replay memory.

- ✓ Sample a mini-batch of experiences from the replay memory.
- ✓ Update the Q-network by minimizing the loss between the predicted Q-values and the target Q-values.
- ✓ Update the target network periodically to stabilize learning.
- ✓ Repeat until convergence or a fixed number of iterations.

Here's an example of implementing DQN using the TensorFlow library in Python:

```
import tensorflow as tf
import numpy as np
import gym

# Create the Q-network model
model = tf.keras.Sequential([
    tf.keras.layers.Dense(32, activation='relu', input_shape=(state_size,)),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(action_size, activation='linear')
])

# Define the loss function and optimizer
loss_fn = tf.keras.losses.MeanSquaredError()
optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)

# Replay memory
memory = []
```

DQN parameters

gamma = 0.99 # discount factor

epsilon = 1.0 # exploration rate

epsilon_min = 0.01

epsilon_decay = 0.995

batch_size = 32

DQN training

for episode in range(num_episodes):

state = env.reset()

done = False

total_reward = 0

while not done:

Epsilon-greedy exploration

if np.random.rand() <= epsilon:

action = env.action_space.sample()

else:

q_values = model.predict(state[np.newaxis])

action = np.argmax(q_values[0])

next_state, reward, done, _ = env.step(action)

total_reward += reward

```

memory.append((state, action, reward, next_state, done))

state = next_state

# Experience replay

if len(memory) >= batch_size:

    batch = random.sample(memory, batch_size)

    states, actions, rewards, next_states, dones = zip(*batch)

    targets = rewards + gamma * (1 - np.array(dones)) *
np.amax(model.predict_on_batch(np.array(next_states)), axis=1)

    with tf.GradientTape() as tape:

        q_values = model(np.array(states))

        selected_action_values = tf.reduce_sum(q_values * tf.one_hot(actions,
action_size), axis=1)

        loss_value = loss_fn(targets, selected_action_values)

        grads = tape.gradient(loss_value, model.trainable_variables)

        optimizer.apply_gradients(zip(grads, model.trainable_variables))

    if epsilon > epsilon_min:

        epsilon *= epsilon_decay

    print(f"Episode: {episode}, Total Reward: {total_reward}")

# Evaluate the trained model

total_rewards = []

for _ in range(num

```

The output of the code will vary depending on the specific problem being solved and the environment used. However, here is an example of the possible output during the training process:

Episode: 0, Total Reward: 100

Episode: 1, Total Reward: 120

Episode: 2, Total Reward: 90

...

Episode: 99, Total Reward: 110

The output shows the episode number and the total reward obtained in each episode during the training process. The actual values may differ based on the specific problem and environment.

During the evaluation phase, the code will print the average reward obtained over a certain number of evaluation episodes:

Average Reward: 115.5

This indicates the average reward achieved by the trained model in the evaluation phase.

Applications:

Deep Reinforcement Learning (DRL) has been successfully applied to various domains and has achieved remarkable results. Here are a few notable applications:

- ✓ **Game Playing:** DRL has been used to train agents to play complex games, such as Atari games and Go. The famous AlphaGo algorithm is an example of DRL applied to the game of Go.

- ✓ Robotics: DRL has been employed to train robots to perform complex tasks, such as grasping objects, locomotion, and autonomous navigation. It enables robots to learn directly from sensory inputs to accomplish specific goals.
- ✓ Autonomous Driving: DRL has been applied to train autonomous vehicles to make decisions in dynamic environments. It helps in learning complex driving policies from visual input and improves decision-making capabilities.
- ✓ Natural Language Processing (NLP): DRL has been utilized in NLP tasks, including dialogue systems and language generation. It enables agents to learn dialogue policies and generate natural language responses.

FOR AUTHOR USE ONLY